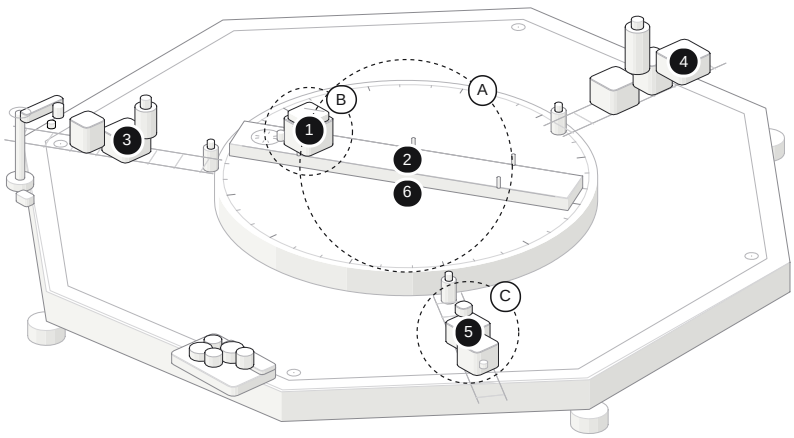
Anywhere



What if the web server were a decision you could defer?

The same application couples to any of three engines through one IServerHost: the threaded Internal engine (HTTP/1.1, embedding), Kestrel (the ASP.NET edge), and Ioxide (io_uring on Linux, HTTP/1.1 to HTTP/3). The meta-packages differ only in the engine dependency, and every acceptance test runs against every engine.

3 engines
INTERNAL, KESTREL, IOXIDE

2 faces
ISERVERHOST ABOVE, ISERVER BELOW

1 hook
EACH ENGINE OVERRIDES A SINGLE BUILD(...)

1 NuGet id
TO SWITCH — THE META-PACKAGES DIFFER ONLY THERE

0 code changes
TO MOVE THE APPLICATION BETWEEN ENGINES

PARTS

01 **The coach**
Your application, unchanged, coupled to whichever engine faces the road.

02 **The turntable**
IServerHost: one coupling face shared by every engine.

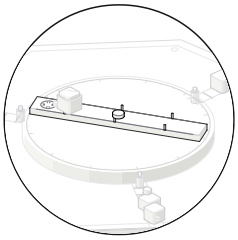
03 **Internal**
Managed sockets, thread-per-connection, HTTP/1.1 — for embedding.

04 **Kestrel**
A hosting engine: ASP.NET Core does transport, GenHTTP keeps handlers.

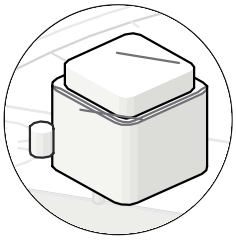
05 **Ioxide**
io_uring per core on Linux, HTTP/1.1 to HTTP/3.

06 **The pit**
The inspectable core below: endpoints, logging, the root router.

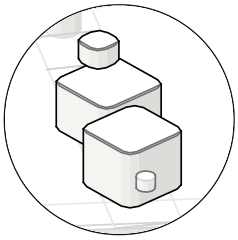
CLOSE-UPS



Ⓐ **The deck**
ServerHost collects endpoints and concerns, applies defaults, assembles the root CoreRouter — then calls the engine's Build.

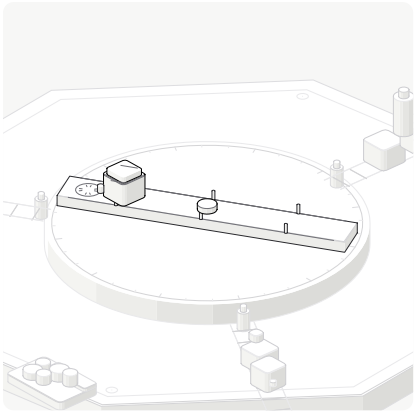


Ⓑ **The coach**
Handlers, concerns and responses are engine-independent by construction; websockets and SSE are specified at the response level.



Ⓒ **The Ioxide road**
CI loads the tls module for kTLS and libmsquic for HTTP/3, so the native path is exercised, not skipped.

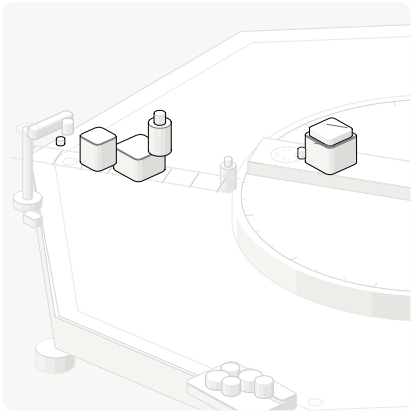
How it works



01

One contract, two faces

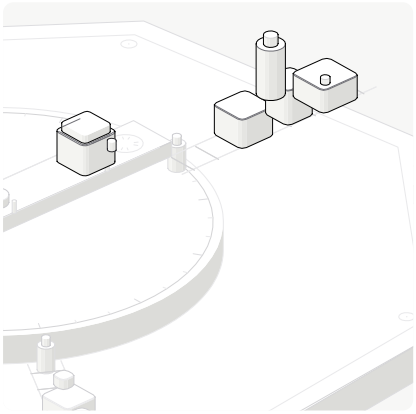
IServerHost is the public hosting API; IServer is what it builds. Each engine overrides a single Build(...) hook — your code does not change.



02

Coupled: Internal

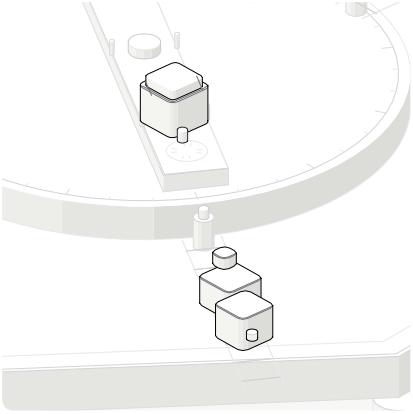
Thread-per-connection managed sockets with Glyph11 parsing: the engine a desktop app embeds, or a service behind a reverse proxy.



03

Coupled: Kestrel

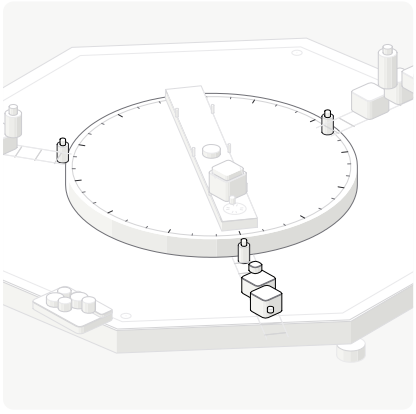
A real WebApplication is built and every HttpContext is bridged into the GenHTTP pipeline — Kestrel does transport, GenHTTP owns responses.



04

Coupled: Ioxide

Native io_uring on Linux, reactors per core, HTTP/1.1 through HTTP/3 — the same handlers, a different kernel story.



05

Proven, not promised

MultiEngineTest runs every wire-level acceptance test against every engine, and CI turns on io_uring, kTLS and libmsquic to exercise the native path.

WHY THREE ENGINES?

The right engine is a deployment decision: embedded inside a desktop app, on the ASP.NET edge of an existing estate, or on bare Linux with io_uring. Because all three keep one contract — and a wire-level test suite fails if any of them drifts — the decision stays deferrable and reversible without touching application code.

SPECIFICATION

Contract	IServerHost / IServer	Ioxide	GenHTTP.Core.Ioxide — io_uring, Linux
Skeleton	Engine/Shared/Hosting/ServerHost.cs	Bridge	HttpContext to IRequest, both directions
Internal	GenHTTP.Core — embed, HTTP/1.1	Tested	every acceptance test, every engine
Kestrel	GenHTTP.Core.Kestrel — the ASP.NET edge		